Jacob Karam, Jack Billington
CSCI 420 - Project 1

Report

For this project, we were given batches of photos taken by Doctor Kinsman on a hike. The goal was to find if each image contained trail markers, and if so put an orange circle around them. To do this, we had to take each image, check if they were good enough to use, then search for the markers. We then output the preprocessed images next to the images with the circled trail markers if applicable.

We were given four batches of fifty images, some of them containing no trail markers and some containing trail markers at various angles and distances. The first step was to read in all of the images. To do this, we had to get the path of the folder containing all of the images. Using a combination of the functions dir() and fullfile(), we accomplish this. After, we run the images through our preprocessing function. The preprocessing function is responsible for preparing each photo so that trail markers can be reliably detected. It removes unusable images, corrects orientation, normalizes exposure, and enhances color contrast before circle detection. There are many photos that we do not want in our data set. Some of these include images that are over-exposed, images that are under-exposed, images that are blurry, and HDR images.

The first thing we do in our preprocessing function is check to see if the image is in the correct orientation. We did this to make sure all the images were right side up so we could more easily compare the results. To do this, we use imfinfo() on the image to get the image's orientation. In the image's metadata, orientation is saved as an integer, 1 being upright, 3 being rotated 180 degrees, 6 being rotated 270 degrees (or -90 degrees), and 8 being rotated 90 degrees. Once we had the image in the correct orientation, the next step was to check if the image's exposure was acceptable.

We noticed while testing that any photos that were over or under-exposed could not find the trail markers, because of the lighting differences. We decided the best way to avoid this problem was to remove these images from our data set, since it added too much variation to the images. To do this, we converted the image to grayscale, then converted the grayscale image to doubles, to get the values of each pixel. Once we had all of the pixels as double values, we would sum all of the pixel values, and divide by the total number of pixels in the image to get the average pixel value over the entire image. If the average pixel value was below a certain threshold, that we set at 0.35 through trial and error, then the image was considered under-exposed, or too dark for us to find the trail markers in. We also had an upper bound to our threshold, which was 0.55, and if the average pixel value was above this value, we considered the photos over-exposed and would also remove these images from our data set.

We also wanted to remove any images from our data set that were blurry. Since we wouldn't be able to accurately locate circles if there were no clear edges, we could not have included any images in which the camera was moving. To do this, we ran a laplacian filter over the image, and checked the variance to see if there were any edges detected. If the images did not have many strong edges, we would throw the images out of the data set. The code we used to remove blurry images also worked to remove any HDR images. This is because of how HDR images are made. It is hard for the laplacian filter to differentiate between edges and texture or noise, so the variance from a laplacian filter on an HDR image would be similar to a blurry image. We were okay with throwing out HDR images because our circle detector could never find the trail markers on these images.

Once we were done removing all the images we did not want in our data set, the next step was to normalize the exposure of images to get them ready to search for the trail markers. To normalize the image, the first thing we did was convert it to the LAB colorspace and separate each of the color channels. The L channel in LAB color space is the luminance channel. This channel is responsible for all of the brightness of the image. The exposure on each image is not the same, so we want to normalize the brightness of each image to make them the same. The reason we normalize the exposure on these images and not the ones that we previously threw out is because there is still a chance to find the markers in these. The previously under or over exposed images have too much information loss and no amount of

normalizing will be able to get it back. We normalize this using histogram equalization. Once that happens, we concatenate the channels back together and convert the image back to RGB. Finally, we run a bilateral filter over the image to smooth it which gets rid of any unwanted artifacts we do not want to detect or mess with.

After preprocessing our valid images, we created a mask using our yellowMask function. This function creates a binary mask of the image in which all yellow regions appear white and everything else appears black, making it much easier to detect circles in the image. To accomplish this, we first converted each image into the LAB color space. Using the A* and B* channels, we created a mask by selecting all pixels whose B* values were greater than 45 and whose A* values were between –15 and 35. We arrived at these ranges after extensive trial and error, determining that they best removed colors that did not match our trail markers. The B* channel represents the yellow-to-blue axis. Yellow has very high B* values, so using only values above 45 effectively eliminates most irrelevant colors. The A* channel represents the green-to-red axis. Yellow typically has a value near zero but slightly positive. Limiting A* to the range –15 to 35 allowed us to include true yellow pixels while removing those that were only slightly yellow but tinted toward red or green.

Once this initial mask was created, we applied a series of morphological operations to better define the circles and eliminate noise caused by stray yellow pixels in the background. We first used morphological closing, which performs dilation followed by erosion. This helped fill small gaps along the edges of the circles that the thresholding step occasionally missed. We then used imfill() to fill in the interiors of the circles, since the trail markers often contain black text or images that would otherwise appear as holes in the mask. After filling the circles, we applied morphological opening, which performs erosion followed by dilation. This removed much of the remaining noise created by unwanted yellow pixels in the background. Because opening can leave small white specks behind, we used bwareaopen() with a threshold of 3000 pixels to remove all connected white regions smaller than that size, effectively eliminating the remaining noise.

With the completed mask, we used imfindcircles() to locate the centers and radii of any trail markers present in each image. After testing various parameters, we set the minimum circle size to 60 pixels to ensure that small noise artifacts would not be detected as circles. For each circle found, we plotted an orange circle at the detected center using the corresponding radius. If an image contained no trail markers or if the mask failed to display them, we still displayed the image but labeled it with the title “No markers found.”

We used ADK_Images_Batch_B to train and refine our preprocessing and masking functions. It contained the widest variety of situations we needed to handle so it made it a good choice for training. This batch had a strong mix of images with and without trail markers, including blurry photos and images that were over-exposed and under-exposed. This variation gave us the best opportunity to test every part of our algorithm, from orientation correction, to exposure filtering, to circle detection. To begin developing our solution, we intentionally started with the simplest possible case. We selected an image that contained a very clear trail marker centered in the frame, with little to no other yellow in the background. This allowed us to establish a baseline that our yellowMask function could detect an obvious marker under ideal conditions. Once we confirmed that our basic mask could isolate the marker and that imfindcircles() could locate it, we used harder images. The next stage was choosing images that still had clear trail markers, but also contained more yellow elsewhere in the scene. This forced us to refine our LAB thresholds, test our morphological operations, and adjust our filtering so that background noise did not appear as false circles. Each time we succeeded on a new “level” of image difficulty, we would try a harder image and update our code to make it work. Over time, this approach helped us develop each component of our code to work properly. By repeatedly testing, adjusting, and scaling up the challenge of the images we used, we were able to grow our solution step by step without needing the entire algorithm completed beforehand.

There were many challenges we faced while working on this project. Our biggest challenge by far was detecting the circles from our mask. The solution that we landed on was one of many different attempts that we tried. One problem we had was trying to figure out how we can fix images that were over

and under exposed. Some of the images, like you see here in Figure 1, are very over-exposed. To try and fix this, we created a new file called homomorph.m. This file created a homomorphic filter that we could run over the image to try and fix the exposure differences. We used it on our images and found that it was messing with our mask a lot. The filter added a ton of noise that our mask was picking up and made it unusable. After doing some more research, we saw that when images are over or under exposed, no matter what you do to them, the information that has been lost in the exposure is already gone. So, we decided to scrap that idea and homomorph.m is no longer in our solution. Another problem we had was tweaking our mask so that it would detect things properly. We tried many different approaches. We tried to use the HSV colorspace, the LAB colorspace, and even a combination of both. Ultimately, we decided to keep LAB. But, we now needed to tweak the thresholds to only pick up the trail markers. We tried many different combinations of opening, closing, filling, and eroding in combination with regionprops(). We did not fully understand how to use regionprops() and after many hours of tweaking, we dropped using it all together. All of this effort was to get our mask correct because our imfindcircles() was detecting so many circles in the image. Eventually, we got the mask to what it is now, and there were still so many circles being found. The circles would be found in the small white noise around the mask and even *INSIDE* the trail markers themselves. We were very confused because even in a situation like Figure 2, there were multiple circles being drawn within those trail markers in Figure 2. It was here that we realized that our mask was not the problem, it was the parameters we were passing into imfindcircles(). The circles it was drawing were very small inside each trailmarker. Seeing this, we changed the range of the radius we were passing in. Before spending hours on the mask, we had tried changing the range of the radius little by little but there was no luck. It was not until this realization that we thought to change the radius but a large amount. We went from [20 60] to [20 200] and our problem had been solved. From there we tweaked the radius to be [60 200] and that is where it sits now.



Figure 1

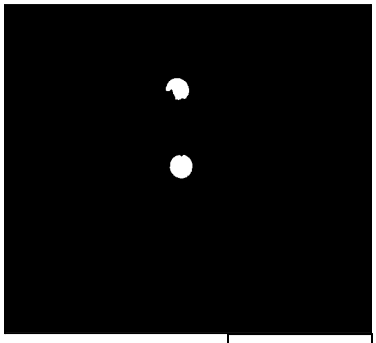


Figure 2

Even with our refined trail marker detector, we still could not identify circles 100% of the time. In Figure 3 below, the circles are all the way on the right side of the image. They are being blocked by leaves and have a ray of light on them. We could not see the trail markers ourselves the first time we looked at this image so there was no way our program was going to be detecting those.



Figure 3

In Figure 4 below, the circles are very close to the screen and it is very blurry and, as you can see, no markers were found.



Figure 4

These images both have one thing in common, they are both at opposite ends of the distance extreme. Depending on where an object is in the camera when a photo is taken, it may appear bigger or smaller. It appears smaller when it's farther away and bigger when it is closer. This is a simple perspective change that completely breaks our program. The radius range we gave imfindcircles() does not account for either of these situations. The perspective problem is not the only thing wrong with our code. Here in Figure 5, the “O” in the word “TOILET” is being detected. This is obviously not a trail marker but our program detects it anyway.



Figure 5

All of these problems are things that we gave up on. We determined that if someone was using this to accurately detect trail markers while on a hike, they would take a clear picture of the trail markers from close, but not too close, distance. If they do this, then it will work perfectly like it does in Figure 6 below. As for the “O” in “TOILET”, it technically is a yellow circle on a dark background, so we chalked it up to a happy accident that happens when you try to push the bounds of what something can do.



Figure 6

To test our program, we split all of the data we were given 25%/75%. The data was four folders of fifty images. Using one of the folders for our training made sense here. The confusion matrix below is a combination of valid images from ADK_Images_Batch_A, ADK_Images_Batch_C, and ADK_Images_Batch_D. From these three folders, our preprocessing function only deemed 53 images as valid. Our confusion matrix below shows the results of our program on those 53 images.

Testing Data: 53 Images Total		True Classification	
		Has Marker	Has No Marker
Classifier Guess:	Has Marker	8	1
	Has No Marker	10	34

Every single trail marker picture that we saw had two trail markers present. Inside of the cell with 10, there are multiple images where only one marker was identified instead of two. We marked these as the classifier guessing no marker. Overall, the classifier worked well. The only false alarm we detected was the “O” from Figure 5 and a little under half of our false negatives were images that detected one, but not both trail markers.

In conclusion, this project taught us to take the concepts and techniques we studied in class and apply them to real world situations. It is said that the most time consuming part of any real world project you do with data is spent cleaning and removing it. This project was no different. We had so much data given to us and in order to build a program that does one thing, consistently and reliably, we had to find out where it is weakest and strongest. Our program perfectly detects yellow hiking trail markers when the picture is taken from a reasonable distance with little to no obstructions and good lighting. Otherwise, it does not do so well. You can take anything that has ever been designed and push it to a limit to make it not work anymore and our program is no different. While not perfect, it taught us many valuable lessons. In the real world, data is very messy and we need to make sure that we remove any outliers and clean it so that we can develop programs that work for a certain purpose. This classifier would not benefit from having images in the data where you can barely see the trail marker in horrible lighting or exposure. If the image is “perfect” and it does not work then there is a problem. This project reinforced the importance of learning how to work in a team to solve a complicated issue. We could not focus all of our effort into this project because of every other class and assignment we had to do, so learning how to manage our time effectively and divide the work was crucial. This project also showed us how to apply what we learned in class, like morphology, to a real world situation. Although not perfect, our project focuses on making one thing so close to perfect that nobody can tell the difference and we believe that to be the most important thing.